

Chapter 8. Iteration 1 Basics

“The hard and stiff breaks. The supple prevails.”

Objectives

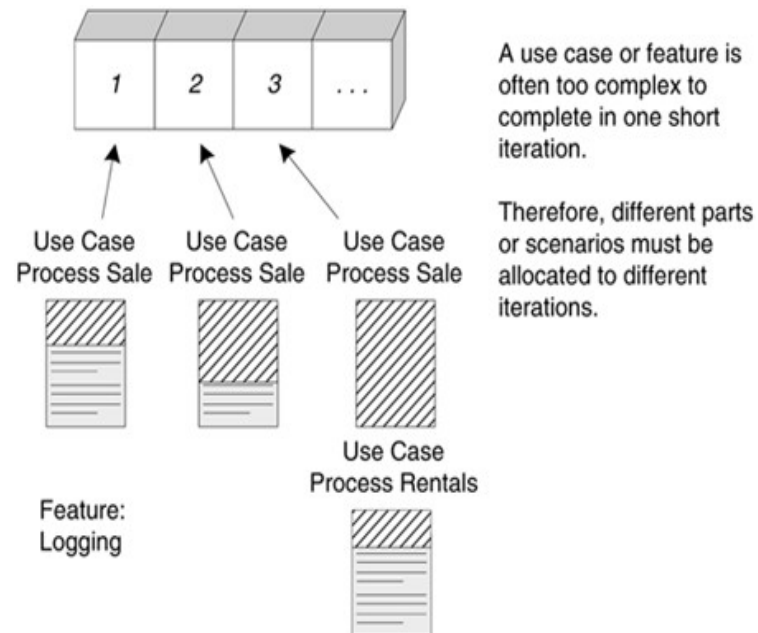
- Define the first iteration in the elaboration phase.
- Motivate the following chapters in this section.
- Describe key inception and elaboration phase concepts.

Iteration 1 Requirements and Emphasis

- 1st Iteration for NextGen POS (in the textbook)
 - Implement a basic, **key scenario** of the *Process Sale use case*
 - (1) entering items
 - (2) receiving a cash payment.
 - Implement a *Start Up use case* : to support the initialization
 - just a simple happy path scenario, and the design and implementation
 - No collaboration with external services (e.g. a tax calculator or database)
 - No complex pricing rules

In Iterative Development, We Do Not Implement All the Requirements at Once

- The requirements for iteration-1 are subsets of the complete requirements or use cases.
- **Incremental development** for the same use case across iterations
 - It is common to work on varying scenarios of the same use case over several iterations and gradually extend the system to ultimately handle all the functionality required.
 - **Short and simple use cases** may be completed within one iteration.



What Happened in Inception?

- Inception is a short (only one week) **step** to elaboration including:
 - A short requirements workshop
 - Most actors, goals, and use cases named
 - Most use cases written in brief format (10~20% are written in fully dressed detail)
 - Most influential and risky requirements identified
 - Version one of the Vision and Supplementary Specification written
 - Risk list
 - Technical proof-of-concept prototypes and other investigations to explore the technical feasibility of special requirements
 - User interface-oriented prototypes to clarify the vision of functional requirements
 - Recommendations on what components to buy/build/reuse, to be refined in elaboration
 - High-level candidate architecture and components proposed
 - Plan for the first iteration
 - Candidate tools list

On to Elaboration

- **Elaboration** is the initial series of iterations during which:
 - The core, risky software architecture is programmed and tested.
 - The majority of requirements are discovered and stabilized.
 - The major risks are mitigated or retired.

Build the core architecture, resolve the high-risk elements, define most requirements, and estimate the overall schedule and resources.

- Some key ideas and best practices
 - Do short timeboxed risk-driven iterations
 - Start programming early
 - Adaptively design, implement, and test the core and risky parts of the architecture
 - Test early, often, realistically
 - Adapt based on feedback from tests, users, developers
 - Write most of the use cases and other requirements in detail, through a series of workshops, once per elaboration iteration

○ Elaboration 단계에서 시작하는 artifact들.

What UP Artifacts May Start in Elaboration?

- Note: these will not be completed in one iteration; rather will be refined over a series of iterations.

↳ iteration을 통해 점점 완성.

Artifact	Comment
Domain Model	This is a visualization of the <u>domain concepts</u> ; it is similar to a static information model of the domain entities.
Design Model	This is the set of diagrams that describes <u>the logical design</u> . This includes <u>software class diagrams</u> , <u>object interaction diagrams</u> , <u>package diagrams</u> , and so forth.
Software Architecture Document	A learning aid that summarizes the <u>key architectural issues</u> and <u>their resolution</u> in the design. It is a summary of the outstanding design ideas and their motivation in the system.
Data Model	This includes the database schemas, and the mapping strategies between object and non-object representations.
Use-Case, UI Prototypes	A description of the user interface, paths of navigation, usability models, and so forth.

You Know You Didn't Understand Elaboration When

- It is more than "a few" months long for most projects.
- It only has one iteration (with rare exceptions for well-understood problems).
- Most requirements were defined before elaboration.
- The risky elements and core architecture are not being tackled.
- It does not result in an executable architecture; there is no production-code programming.
- It is considered primarily a requirements or design phase, preceding an implementation phase in construction.
- There is an attempt to do a full and careful design before programming.
- There is minimal feedback and adaptation; users are not continually engaged in evaluation and feedback.
- There is no early and realistic testing.
- The architecture is speculatively finalized before programming.
- It is considered a step to do the proof-of-concept programming, rather than programming the production core executable architecture.

◦ iteration은 고정길이 2-6주 ◦ Elaboration 초기 iteration에는 UML과 OOA/D 집중.
◦ 기간을 맞추기 어려우면 작업범위를 줄이고 다음 iteration으로 이동(de-scope)

Use Case Ranking

- Organize requirements and iterations by
 - Risk** : includes both technical complexity and other factors such as uncertainty of effort or usability. *→ 2차 먼저 다루 필요↑*
 - Coverage** : implies that all major parts of the system are at least touched on in early iterations; a “wide and shallow” implementation across many components.
 - Criticality** : refers to functions the client considers of high business value. *→ 비즈니스 가치.*
- Use cases are ranked for implementation.
 - Early iterations will implement high ranking use cases and use case scenarios.

Rank	Requirement (Use Case or Feature)	Comment
High	Process Sale	Scores high on all rankings.
	Logging	Pervasive. Hard to add late.
	...	...
Medium	Maintain Users	Affects security subdomain.
	...	...
Low	...	...